

Міністерство освіти і науки України
Харківський національний університет радіоелектроніки

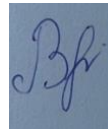
Кафедра програмної інженерії

КУРСОВА РОБОТА
Пояснювальна записка

З дисципліни «Об'єктно-орієнтоване програмування»
«Довідник лікаря»

Виконала:

здобувачка 1 року навчання , групи ПЗПІ-25-1



Марія РИЖОВА

(власне ім'я , ПРИЗВИЩЕ)

Керівник Проф.Володимир БОНДАРЄВ

(підпис) (посада ,власне ім'я,ПРИЗВИЩЕ)

Комісія:

Проф.Володимир БОНДАРЄВ

Ст.викл.Юлія ЧЕРЕПАНОВА

Ст.викл.Віталій ЛЯПОТА

Харків 2026

ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра. *програмної інженерії*
Рівень вищої освіти. *перший (бакалаврський)*
Дисципліна. *Об'єктно-орієнтоване програмування*
Спеціальність *121 Інженерія програмного забезпечення*
Освітня програма. *Програмна інженерія*
Курс. 1. Група ПЗПІ-25 -1. Семестр 2.

ЗАВДАННЯ

на курсову роботу здобувачеві

Рижова Марія Володимирівна

(Прізвище, Ім'я, По батькові)

1 Тема роботи

Розробка програми Довідник лікаря за допомогою С#

2 Термін подання здобувачем роботи до захисту **“30” - травня - 2026 р.**

3 Вихідні дані до роботи

Довідник лікаря. База хвороб: назва, симптоми, процедури, перелік рекомендованих ліків із зазначенням необхідної кількості. База медикаментів на складі: назва, кількість, взаємозамінність. Формування рецепта після огляду хворого, перевірка наявності ліків, коригування запасів

4 Перелік питань, що потрібно опрацювати в роботі

Вступ, опис вимог, проєктування програми, інструкція користувача,

Висновки

КАЛЕНДАРНИЙ ПЛАН

№	Назва етапів роботи	Термін виконання
1	Видача теми, узгодження і затвердження теми (1 тижд.)	25.03.2026 - 01.04.2026 р.
2	Формулювання мети роботи (1 тижд.)	01.04.2026 – 08.04.2026 р.
3	Опис вимог до програми (1 тижд.)	08.04.2026 – 15.04.2026 р.
4	Проєктування програми (1 тижд.)	15.04.2026 – 22.04.2026 р.
5	Кодування (2 тижд.)	22.04.2026 – 06.05.2026 р.
6	Підготовка роботи до захисту (1 тижд.)	13.05.2026 – 20.05.2026 р.
7	Захист	20.05.2026 – 30.05.2026 р.



Здобувач _____

Керівник _____ Проф.Володимир БОНДАРЄВ
(посада, Ім'я, ПРІЗВИЩЕ)

« 21 » _лютого_____ 2026 р.

РЕФЕРАТ

Пояснювальна записка до курсової роботи на тему «Довідник лікаря»: 38 с., 6 рис., 5 табл., 4 джерела посилання, 2 додатки.

Об'єкт розроблення: програмна система обліку хвороб, медикаментів та виписування рецептів лікарем.

Мета роботи: проєктування та реалізація об'єктно-орієнтованої програми «Довідник лікаря» засобами мови C# та платформи Windows Forms, яка автоматизує роботу лікаря з базами хвороб і медикаментів, формує рецепти та відстежує залишки на складі.

Методи дослідження: об'єктно-орієнтований аналіз та проєктування, UML-моделювання, патерн Singleton, серіалізація даних у форматі JSON.

КЛЮЧОВІ СЛОВА: Довідник лікаря, Windows Forms, C#, .NET 6, база хвороб, медикаменти, рецепт, JSON, UML, ООП.

ЗМІСТ

Вступ	7
РОЗДІЛ 1. АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ФУНКЦІОНАЛЬНИХ ВИМОГ	8
1.1. Опис бізнес-процесів та діяльності лікаря	9
1.2. Специфікація сценаріїв використання (Use Cases)	10
1.2.1. Сценарій автоматичного формування рецепта за симптомами	11
1.2.2. Сценарій коригування та обліку запасів медикаментів	12
1.3. Проектування макетів користувацького інтерфейсу	13
РОЗДІЛ 2. ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА ОБ'ЄКТНОЇ МОДЕЛІ	14
2.1. Логічна структура та модульна архітектура проєкту	16
2.2. Об'єктно-орієнтований аналіз та проектування класів системи	17
2.2.1. Базові класи-сутності даних (Disease, Medication)	18
2.2.2. Сервісні класи бізнес-логіки (WarehouseService, DiagnosisService) ...	19
2.3. UML-діаграма класів системи	20
2.4. Математично-алгоритмічне забезпечення роботи системи	21
2.4.1. Алгоритм підбору замінників ліків за дефіциту на складі	23

РОЗДІЛ 3. ІНСТРУКЦІЯ З ЕКСПЛУАТАЦІЇ ТА РОЗГОРТАННЯ
ПРОГРАМИ 24

3.1. Системні вимоги до апаратного та програмного забезпечення
..... 25

3.2. Покрокове керівництво з інсталяції та запуску проєкту
..... 26

3.3. Рекомендації з деінсталяції та резервного копіювання
..... 27

Висновки 28

Перелік джерел посилання
..... 30

Додаток А. Вихідний код програми (C#)
..... 31

Додаток Б. Графічні матеріали та блок-схеми алгоритмів
..... 34

ВСТУП

Сучасна медицина потребує ефективних програмних інструментів для автоматизації рутинних процесів: обліку хвороб, управління запасами медикаментів, формування рецептів і відстеження залишків на складі. Лікар щоденно стикається з необхідністю швидко знаходити інформацію про хвороби, перевіряти наявність ліків і виписувати рецепти, дотримуючись при цьому протоколів лікування. Відсутність зручного автоматизованого інструменту призводить до помилок, затримок і неефективного використання часу медичного персоналу.

Об'єктом розроблення є програмна система «Довідник лікаря», яка забезпечує ведення бази хвороб із симптомами, процедурами та рекомендованими медикаментами; управління складом ліків із контролем залишків і пошуком замінників; формування рецептів після огляду пацієнта з автоматичним списанням медикаментів зі складу.

Метою роботи є проєктування та реалізація об'єктно-орієнтованої програми «Довідник лікаря» засобами мови програмування C# та технології Windows Forms на платформі .NET 6. Програма повинна підтримувати повний цикл CRUD-операцій над базами даних хвороб і медикаментів, реалізовувати пошук і фільтрацію, а також автоматизувати процес формування рецептів із перевіркою наявності ліків.

Практична цінність роботи полягає у тому, що розроблений програмний продукт може бути використаний як навчальний прототип системи управління медичними даними у невеликому медичному закладі або приватній практиці лікаря.

Для досягнення поставленої мети в роботі вирішено такі завдання:

- проаналізовано предметну область та визначено функціональні вимоги до програми;
- розроблено сценарії використання (Use Cases) та ескізи інтерфейсу;
- спроектовано об'єктну модель системи з використанням UML-діаграм класів;
- реалізовано програму мовою C# з використанням Windows Forms та серіалізацією у JSON;
- розроблено інструкцію з встановлення та використання програми.

РОЗДІЛ 1

АНАЛІЗ ПРЕДМЕТНОЇ ОБЛАСТІ ТА ФУНКЦІОНАЛЬНИХ ВИМОГ

1.1. Опис бізнес-процесів та діяльності лікаря

Автоматизована система «Довідник лікаря» призначена для підтримки щоденної клінічної роботи лікаря загальної практики. Ключовим завданням є скорочення часу на рутинні операції пошуку інформації та виписування рецептів, а також мінімізація помилок, пов'язаних із людським чинником при роботі з великими обсягами медичних даних.

Типовий робочий процес лікаря складається з кількох взаємопов'язаних бізнес-процесів. Перший процес — ведення бази хвороб. Лікар повинен мати швидкий доступ до довідкової інформації про захворювання: їх назви, характерні симптоми, рекомендовані процедури та перелік ліків із необхідним дозуванням. Ця інформація слугує основою для прийняття клінічних рішень.

Другий бізнес-процес — управління складом медикаментів. Медичний заклад постійно оперує запасами ліків: поповнює склад, списує ліки при видачі пацієнтам, контролює строки придатності та шукає заміники при відсутності основного препарату. Точний облік залишків критично важливий для ефективного лікування.

Третій процес — огляд пацієнта та виписування рецепта. Після встановлення діагнозу лікар підбирає схему лікування, перевіряє наявність необхідних медикаментів на складі і формує рецепт. Система повинна автоматично перевіряти наявність кожного ліку і пропонувати заміники при їх відсутності.

Таблиця 1.1 — Перелік бізнес-процесів та їх автоматизація

№	Бізнес-процес	Проблема без автоматизації	Рішення у програмі
1	Ведення бази хвороб	Ручний пошук у паперових довідниках, повільно та ненадійно	Швидкий пошук за назвою та симптомами, CRUD-операції
2	Облік медикаментів	Відсутність актуальної інформації про залишки, ризик помилок	Реєстр ліків із кількістю, автоматичне списання
3	Виписування рецептів	Ручне заповнення форм, відсутність перевірки наявності ліків	Автоматичне формування рецепта, перевірка залишків
4	Пошук заміників ліків	Немає систематизованої інформації про взаємозамінні препарати	Поле взаємозамінності, підказки при дефіциті
5	Архів рецептів	Паперовий архів, складний пошук	Електронний архів рецептів із пошуком

Аналіз предметної галузі дозволяє виділити дві основні сутності системи. Перша сутність — хвороба (Disease) — характеризується назвою, переліком симптомів, рекомендованими процедурами та списком необхідних медикаментів із кількістю. Друга сутність — медикамент (Medicine) — описується назвою, кількістю в наявності та переліком взаємозамінних препаратів

Між цими сутностями існує зв'язок «багато-до-багатьох»: одна хвороба може потребувати кількох медикаментів, і один медикамент може використовуватись при різних хворобах. Цей зв'язок реалізується через клас-посередник DiseaseMedicine, який фіксує ідентифікатор медикаменту та необхідну кількість. Третьою ключовою сутністю є рецепт (Prescription) — документ, що формується після огляду пацієнта. Рецепт містить ПІБ пацієнта, ПІБ лікаря, назву діагнозу, дату та список призначених медикаментів із зазначенням їх наявності на складі.

1.2. Специфікація сценаріїв використання (Use Cases)

На основі аналізу бізнес-процесів визначено наступний перелік сценаріїв використання програми:

- 1) Пошук хвороби за назвою або симптомами.
- 2) Додавання нової хвороби до бази.
- 3) Редагування існуючої хвороби.
- 4) Видалення хвороби з підтвердженням.
- 5) Перегляд рекомендованих медикаментів для хвороби.
- 6) Пошук медикаменту за назвою або взаємозамінністю.
- 7) Додавання медикаменту на склад.
- 8) Редагування даних медикаменту.
- 9) Коригування залишків медикаменту.
- 10) Видалення медикаменту.
- 11) Формування рецепта для пацієнта.
- 12) Перевірка наявності ліків при формуванні рецепта.
- 13) Перегляд архіву виписаних рецептів.
- 14) Видалення рецепта з архіву.

Таблиця 1.2 — Специфікація сценаріїв використання

Сценарій	Актор	Передумова	Результат
Пошук хвороби	Лікар	Відкрита вкладка «Хвороби»	Відображено список хвороб за запитом
Додавання хвороби	Лікар	Натиснута кнопка «Додати»	Хвороба збережена у базі
Формування рецепта	Лікар	Введено ПІБ пацієнта та обрано діагноз	Рецепт збережено, ліки списано зі складу
Коригування залишків	Адмін/Лікар	Обрано медикамент у реєстрі	Кількість медикаменту оновлено
Перегляд архіву рецептів	Лікар	Відкрита вкладка «Рецепти»	Відображено список виписаних рецептів

1.2.1. Сценарій автоматичного формування рецепта за симптомами

Даний сценарій є ключовим з точки зору автоматизації роботи лікаря. Він описує процес формування рецепта з автоматичною підстановкою медикаментів та перевіркою їх наявності на складі.

Назва сценарію: «Формування рецепта після огляду пацієнта».

Актор: Лікар.

Передумова: лікар встановив діагноз і перейшов на вкладку «Виписати рецепт».

Постумова: рецепт збережено в архіві, залишки медикаментів на складі зменшено.

Основний сценарій:

- 1) Лікар вводить прізвище, ім'я та по батькові пацієнта у відповідне поле вводу.
- 2) Лікар вводить власне ПІБ у поле «Лікар».
- 3) Лікар обирає діагноз зі спадного списку, сформованого з бази хвороб.
- 4) Програма автоматично формує список рекомендованих медикаментів для обраного діагнозу.
- 5) Програма перевіряє наявність кожного медикаменту на складі та відображає статус: «є в наявності» або «відсутній».
- 6) У правій частині вікна відображається попередній перегляд рецепта.
- 7) Лікар натискає кнопку «Виписати рецепт».
- 8) Програма перевіряє коректність заповнення полів (обов'язкові поля: ПІБ пацієнта, ПІБ лікаря, діагноз).
- 9) Програма списує наявні медикаменти зі складу (кількість у базі зменшується).
- 10) Рецепт зберігається в архіві та доступний для перегляду на вкладці «Рецепти».

Альтернативний сценарій А — не всі ліки є в наявності:

- 1) На кроці 5 програма виявляє, що один або кілька медикаментів відсутні на складі.
- 2) Відповідні позиції у списку позначаються червоним кольором зі статусом «відсутній».
- 3) Програма перевіряє наявність взаємозамінних препаратів у базі медикаментів.
- 4) Якщо заміник знайдено — система відображає інформацію про нього поруч.
- 5) Лікар приймає рішення: виписати рецепт із відсутніми позиціями або відкоригувати список.
- 6) При натисканні «Виписати рецепт» система попереджає про відсутні ліки та запитує підтвердження.
- 7) При підтвердженні рецепт зберігається, доступні ліки списуються.

Альтернативний сценарій Б — некоректне заповнення форми:

- 1) Лікар не заповнив обов'язкові поля та натиснув «Виписати рецепт».
- 2) Програма відображає повідомлення про помилки над полями.
- 3) Вікно залишається відкритим, рецепт не зберігається.

1.2.2. Сценарій коригування та обліку запасів медикаментів на складі

Сценарій описує процес управління залишками медикаментів, що є критично важливим для безперебійної роботи медичного закладу.

Назва сценарію: «Коригування запасів медикаменту».

Актор: Лікар / Завідувач складу.

Передумова: відкрита вкладка «Медикаменти», обрано конкретний медикамент.

Постумова: кількість медикаменту оновлено у базі та збережено.

Основний сценарій (поповнення складу):

- 1) Користувач обирає медикамент зі списку — деталі відображаються у правій панелі.
- 2) У полі «Коригування запасу» вводить позитивне число (наприклад, +50).
- 3) Натискає кнопку «Застосувати».
- 4) Програма додає введену кількість до поточного залишку.
- 5) Список оновлюється, нова кількість відображається у стовпці «В наявності».
- 6) Відображається повідомлення про успішне оновлення.

Альтернативний сценарій — часткове списання:

- 1) Користувач вводить від'ємне значення (наприклад, -10).
- 2) Програма перевіряє, чи не перевищує кількість списання поточний залишок.
- 3) Якщо залишок достатній — виконується списання, кількість зменшується.
- 4) Якщо залишок недостатній — відображається повідомлення про помилку, зміни не зберігаються.

1.3. Проектування макетів користувацького інтерфейсу

Програма реалізована у вигляді настільного застосунку Windows Forms з головним вікном, що містить чотири вкладки (TabControl). Кожна вкладка відповідає окремому функціональному модулю системи. Нижче наведено детальний опис макетів кожної вкладки.

Вкладка «Хвороби» (рис. 1.1) розділена на дві панелі. Ліва панель містить панель інструментів (поле пошуку та кнопки «Пошук», «Додати», «Змінити», «Видалити») та список хвороб (ListBox). Права панель відображає деталі

обраної хвороби: поля «Назва», «Симптоми», «Процедури» (TextBox у режимі лише для читання) та список «Рекомендовані ліки» (ListBox).

Вкладка «Медикаменти» (рис. 1.2) має аналогічну структуру: ліворуч розташовано ListView із двома стовпцями (назва та кількість). Записи з нульовим залишком виділяються червоним кольором, із залишком менше 10 одиниць — оранжевим. Права панель містить деталі медикаменту та блок коригування залишків.

Вкладка «Виписати рецепт» (рис. 1.3) розділена на дві частини. Ліворуч знаходиться форма введення даних: поля «ПІБ пацієнта», «ПІБ лікаря», спадний список «Діагноз» та ListBox зі списком медикаментів рецепта з відміткою їх наявності. Праворуч розміщено вікно попереднього перегляду рецепта (RichTextBox).

Вкладка «Рецепти» (рис. 1.4) містить ListView з архівом виписаних рецептів та правою панеллю для детального перегляду обраного рецепта.

Рисунок 1.1 — Ескіз вкладки «Хвороби»

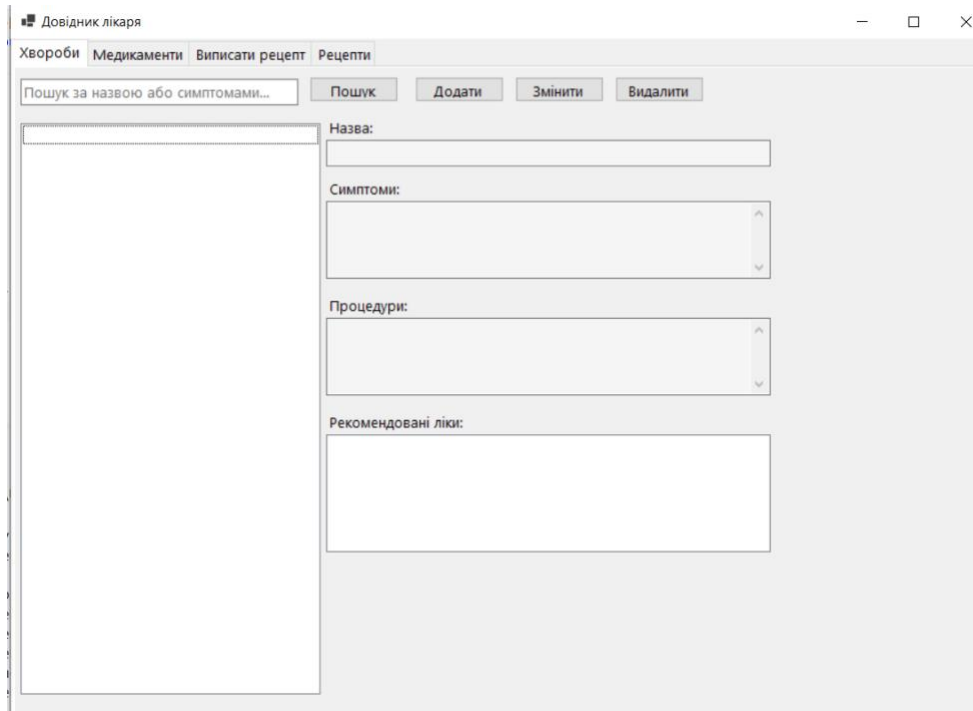


Рисунок 1.2-Ескіз вкладки «Медикаменти»

Пошук за пацієнтом або діагнозом... Пошук Видалити

[illegible]

Деталі рецепта:

--

РОЗДІЛ 2

ПРОЄКТУВАННЯ АРХІТЕКТУРИ ТА ОБ'ЄКТНОЇ МОДЕЛІ

2.1. Логічна структура та модульна архітектура проєкту

Програма «Довідник лікаря» побудована за принципом розподілу відповідальностей (Separation of Concerns). Проєкт поділено на чотири логічні пакети, кожен з яких відповідає за окремий аспект роботи системи. Таке розбиття забезпечує зрозумілість коду, спрощує тестування та подальший супровід.

Таблиця 2.1 — Структура пакетів проєкту

Пакет	Класи	Призначення
Models	Disease, Medicine, DiseaseMedicine, Prescription, PrescriptionItem	Класи-сутності даних предметної галузі. Описують структуру об'єктів без логіки.
Data	DataStore	Патерн Singleton. Централізоване сховище колекцій об'єктів, CRUD-операції, серіалізація/десеріалізація у JSON.
Forms	MainForm, DiseasesPanel, MedicinesPanel, PrescriptionPanel, PrescriptionHistoryPanel, DiseaseEditForm, MedicineEditForm, SelectMedicineForm	Класи Windows Forms. Реалізують графічний інтерфейс користувача, обробляють події та взаємодіють із DataStore.
Program	Program	Точка входу в програму. Налаштовує середовище та запускає головну форму.

Пакет Models містить виключно класи-описи сутностей без будь-якої бізнес-логіки. Це відповідає принципу єдиної відповідальності (Single Responsibility Principle). Кожен клас моделі серіалізується у JSON і зберігається у відповідному файлі.

Клас DataStore реалізує патерн Singleton, що гарантує існування єдиного екземпляра сховища даних у всій програмі. Усі форми звертаються до даних виключно через DataStore.Instance, що запобігає дублюванню даних та забезпечує їх цілісність.

Форми реалізують інтерфейс `IRefreshable`, що дозволяє головній формі автоматично оновлювати вміст вкладки при її виборі. Це гарантує, що дані на кожній вкладці завжди актуальні після змін, внесених на інших вкладках.

Взаємодія між пакетами організована наступним чином: форми звертаються до `DataStore` для читання та модифікації даних; `DataStore` оперує об'єктами класів з пакету `Models`; збереження відбувається через `System.Text.Json` безпосередньо у `DataStore`. Такий підхід мінімізує зв'язність між компонентами.

Таблиця 2.2 — Файли проєкту та їх призначення

Файл	Пакет	Опис
Program.cs	—	Точка входу, налаштування WinForms
Models/Disease.cs	Models	Клас хвороби з переліком ліків
Models/Medicine.cs	Models	Клас медикаменту зі складу
Models/DiseaseMedicine.cs	Models	Зв'язок хвороба — медикамент
Models/Prescription.cs	Models	Рецепт із переліком призначень
Models/PrescriptionItem.cs	Models	Один рядок рецепта
Data/DataStore.cs	Data	Singleton: сховище + CRUD + JSON
Forms/MainForm.cs	Forms	Головне вікно з вкладками
Forms/DiseasesPanel.cs	Forms	Вкладка управління хворобами
Forms/MedicinesPanel.cs	Forms	Вкладка управління медикаментами
Forms/PrescriptionPanel.cs	Forms	Вкладка виписування рецептів
diseases.json	—	Файл збереження бази хвороб
medicines.json	—	Файл збереження бази медикаментів
prescriptions.json	—	Файл збереження архіву рецептів

2.2. Об'єктно-орієнтований аналіз та проєктування класів системи

Об'єктна модель системи побудована відповідно до принципів SOLID. Кожен клас має чітко визначену відповідальність. Спадкування не використовується там, де воно надмірне — натомість застосовується композиція. Всі класи моделей є серіалізованими завдяки відсутності непідтримуваних типів.

Загальна об'єктна модель включає п'ять класів-сутностей і три сервісні/допоміжні класи. Між класами визначено зв'язки агрегації та залежності, що детально описані нижче.

2.2.1. Базові класи-сутності даних (Disease, Medicine)

Клас Disease описує медичне захворювання. Він містить унікальний ідентифікатор, назву, текстовий опис симптомів, перелік рекомендованих процедур та список рекомендованих медикаментів типу `List<DiseaseMedicine>`. Зв'язок з медикаментами реалізований через агрегацію: об'єкт Disease «знає» про необхідні медикаменти, але не «створює» їх.

«class» Disease
+ Id : int
+ Name : string
+ Symptoms : string
+ Procedures : string
+ RecommendedMedicines : List<DiseaseMedicine>
+ Disease()
+ Disease(id, name, symptoms, procedures)
+ ToString() : string

Рисунок 2.1 — UML-діаграма класу Disease

Клас Medicine описує медикамент на складі. Ключовим є поле Quantity, яке відображає поточну кількість у наявності. Поле Alternatives зберігає рядок з переліком назв взаємозамінних препаратів, що використовується при пошуку замінників у разі дефіциту.

«class» Medicine
+ Id : int
+ Name : string
+ Quantity : int
+ Alternatives : string
+ Medicine()
+ Medicine(id, name, quantity, alternatives)
+ ToString() : string

Рисунок 2.2 — UML-діаграма класу Medicine

Клас DiseaseMedicine є класом-посередником, що реалізує зв'язок «багато-до-багатьох» між хворобою та медикаментом. Він зберігає ідентифікатор медикаменту, його назву (для зручності відображення без додаткових запитів) та необхідну кількість для лікування.

«class» DiseaseMedicine
+ MedicineId : int
+ MedicineName : string
+ RequiredQuantity : int
+ DiseaseMedicine()
+ DiseaseMedicine(medicineId, medicineName, requiredQty)
+ ToString() : string

Рисунок 2.3 — UML-діаграма класу DiseaseMedicine

Клас Prescription описує виписаний рецепт. Він агрегує список об'єктів PrescriptionItem, кожен з яких відповідає одному медикаменту у рецепті. Поле Date автоматично встановлюється на поточну дату та час при створенні рецепта.

«class» Prescription
+ Id : int
+ PatientName : string

+ DoctorName : string
+ DiagnosisName : string
+ Date : DateTime
+ Items : List<PrescriptionItem>
+ Prescription()
+ Prescription(id, patientName, doctorName, diagnosisName)
+ ToString() : string

Рисунок 2.4 — UML-діаграма класу Prescription

2.2.2. Сервісні класи бізнес-логіки (DataStore)

Клас DataStore є центральним класом системи, що реалізує патерн проектування Singleton. Він зберігає три публічні колекції: Diseases, Medicines та Prescriptions. Доступ до єдиного екземпляра здійснюється через статичну властивість Instance. Приватний конструктор запобігає створенню додаткових екземплярів.

DataStore надає методи для кожної з колекцій: AddDisease(), UpdateDisease(), DeleteDisease(), SearchDiseases() — для хвороб; аналогічні методи — для медикаментів. Метод AdjustMedicineStock() реалізує перевірку достатності залишків перед списанням. Методи SaveAll() та LoadAll() забезпечують серіалізацію через System.Text.Json.

«class» DataStore
- _instance : DataStore {static}
- _diseasesFile : string = "diseases.json"
- _medicinesFile : string = "medicines.json"
- _prescriptionsFile : string = "prescriptions.json"
- _nextDiseaseId : int
- _nextMedicineId : int
- _nextPrescriptionId : int
+ Diseases : List<Disease>
+ Medicines : List<Medicine>
+ Prescriptions : List<Prescription>
+ Instance : DataStore {static, get}
- DataStore() {private}

+ AddDisease(disease) : void
+ UpdateDisease(disease) : void
+ DeleteDisease(id) : void
+ SearchDiseases(query) : List<Disease>
+ AddMedicine(medicine) : void
+ UpdateMedicine(medicine) : void
+ DeleteMedicine(id) : void
+ AdjustMedicineStock(id, qty) : bool
+ AddPrescription(prescription) : void
+ SaveAll() : void
+ LoadAll() : void
+ SeedSampleData() : void

Рисунок 2.5 — UML-діаграма класу DataStore

2.3. Графічне моделювання структури за допомогою UML-діаграми класів

На рисунку 2.6 наведено зведену UML-діаграму класів системи «Довідник лікаря», що відображає зв'язки між усіма класами. Діаграма демонструє зв'язки агрегації, залежності та реалізації інтерфейсу.

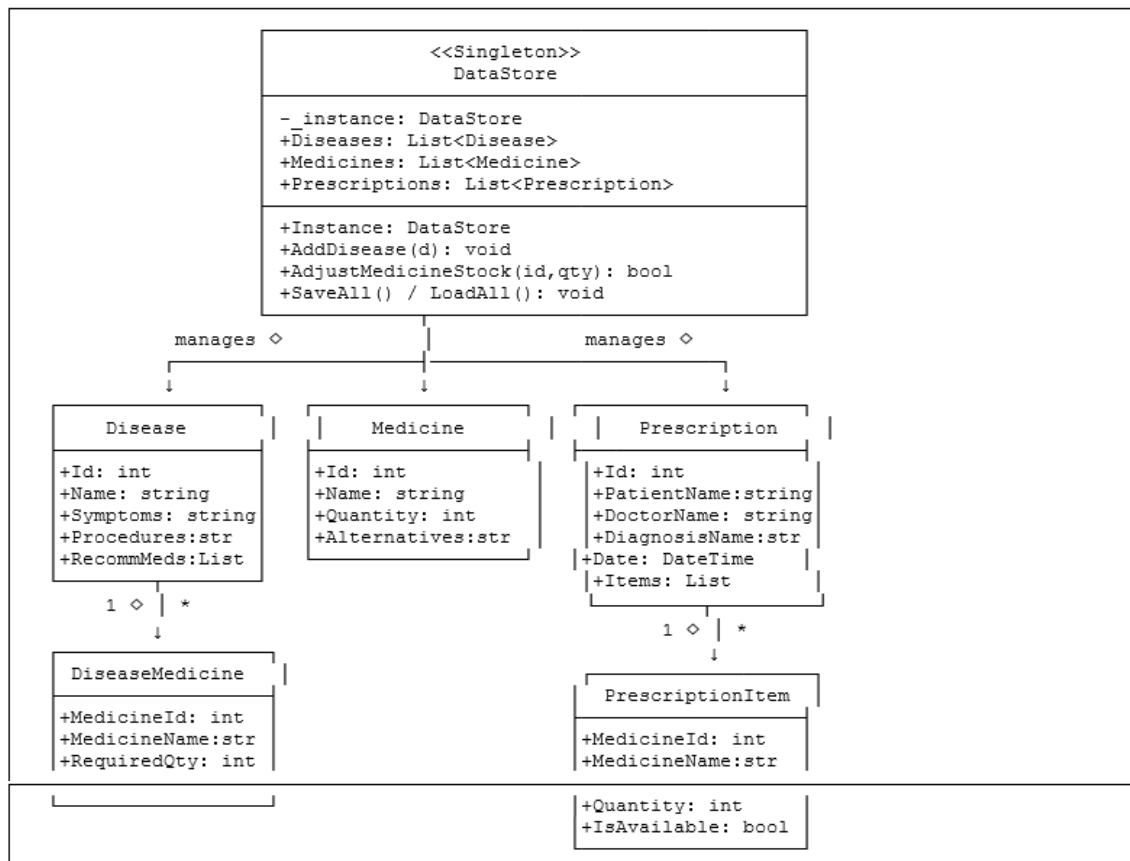


Рисунок 2.6 — Зведена UML-діаграма класів системи «Довідник лікаря»

Аналізуючи діаграму, можна виділити такі ключові зв'язки. По-перше, DataStore агрегує колекції всіх трьох основних класів (Disease, Medicine, Prescription) — цей зв'язок зображено символом «діамант». По-друге, між Disease та DiseaseMedicine існує зв'язок «один-до-багатьох» (один об'єкт Disease може мати список DiseaseMedicine). По-третє, між Prescription та PrescriptionItem аналогічний зв'язок «один-до-багатьох». По-четверте, DiseaseMedicine та PrescriptionItem посилаються на Medicine за ідентифікатором (слабкий зв'язок залежності).

2.4. Математично-алгоритмічне забезпечення роботи системи

2.4.1. Алгоритм підбору замінників ліків за дефіциту на складі

При формуванні рецепта система перевіряє наявність кожного з рекомендованих медикаментів на складі. Якщо кількість медикаменту недостатня або він відсутній, система аналізує поле Alternatives кожного медикаменту та формує список можливих замінників. Нижче описано алгоритм цього процесу.

Вхідні дані: об'єкт Disease з переліком DiseaseMedicine, колекція Medicines у DataStore.

Вихідні дані: список PrescriptionItem із заповненим полем IsAvailable для кожного медикаменту.

Опис алгоритму:

- 1) Для кожного об'єкта dm у Disease.RecommendedMedicines:
 - 2) а. Знайти об'єкт med у Medicines за med.Id == dm.MedicinId.
 - 3) б. Перевірити: якщо med != null та med.Quantity >= dm.RequiredQuantity — встановити IsAvailable = true.
 - 4) в. Інакше IsAvailable = false.
 - 5) г. Сформувати об'єкт PrescriptionItem з відповідним значенням IsAvailable.
- 6) Якщо для будь-якого PrescriptionItem IsAvailable = false:
 - 7) а. Для поточного медикаменту (med) перевірити поле Alternatives.
 - 8) б. Розбити рядок Alternatives на токени за символом «,».
 - 9) в. Для кожного токена знайти медикамент у Medicines за назвою (Contains, без урахування регістру).
 - 10) г. Якщо знайдено замінник із достатнім залишком — відобразити його назву поруч із відсутнім медикаментом.
- 11) Повернути список PrescriptionItem із позначками доступності.

```

List<PrescriptionItem> CheckAvailability(Disease disease)
{
    var result = new List<PrescriptionItem>();
    foreach (DiseaseMedicine dm in disease.RecommendedMedicines)
    {
        Medicine med = Medicines.Find(m => m.Id == dm.MedicineId);
        bool available = med != null && med.Quantity >=
dm.RequiredQuantity;
        var item = new PrescriptionItem(
            dm.MedicineId, dm.MedicineName, dm.RequiredQuantity,
available),
        if (!available && med != null &&
!string.IsNullOrEmpty(med.Alternatives))
        {
            string[] alts = med.Alternatives.Split(',');
            foreach (string alt in alts)
            {
                Medicine altMed = Medicines.Find(
                    m =>
m.Name.ToLower().Contains(alt.Trim().ToLower())
                    && m.Quantity >= dm.RequiredQuantity);
                if (altMed != null) item.AlternativeName = altMed.Name;
break;
            }
        }
        result.Add(item);
    }
    return result;
}

```

Таблиця 2.3 — Складність алгоритму підбору заміників

Параметр	Позначення	Значення / Складність
Кількість ліків у хворобі	k	$O(k)$ — лінійна від k-ті медикаментів
Розмір бази медикаментів	n	$O(n)$ — пошук за ідентифікатором
K-ть заміників у полі	a	$O(a \cdot n)$ — пошук заміників
Загальна складність	T	$O(k \cdot (n + a \cdot n)) = O(k \cdot n \cdot a)$

Оскільки у типовій медичній практиці k (кількість ліків у одній хворобі) зазвичай не перевищує 10, а a (кількість замінників) — 5, загальна складність фактично є квазілінійною від розміру бази медикаментів n . Це забезпечує прийнятну швидкодію навіть при великих базах.

РОЗДІЛ 3

ІНСТРУКЦІЯ З ЕКСПЛУАТАЦІЇ ТА РОЗГОРТАННЯ ПРОГРАМИ

3.1. Системні вимоги до апаратного та програмного забезпечення

Для коректного функціонування програми «Довідник лікаря» необхідно дотриматись мінімальних системних вимог до апаратного та програмного забезпечення комп'ютера. Нижче наведено детальний перелік вимог.

Таблиця 3.1 — Системні вимоги

Компонент	Мінімальна вимога	Рекомендована вимога
Операційна система	Windows 10 (64-bit)	Windows 10/11 (64-bit)
Процесор	Intel Core i3 або еквівалент, 1.6 ГГц	Intel Core i5 або вище, 2.0 ГГц
Оперативна пам'ять	4 ГБ RAM	8 ГБ RAM
Вільне місце на диску	200 МБ (з .NET Runtime)	500 МБ
Дисплей	1280×720 пікселів	1920×1080 та вище
.NET Runtime	.NET 6.0 Runtime (Windows)	.NET 6.0 або .NET 8.0
Microsoft Visual Studio	2022 Community (для розробки)	2022 Professional/Enterprise
.NET SDK	.NET 6 SDK	.NET 6 або .NET 8 SDK

Програмне забезпечення, яке має бути встановлено перед запуском:

- .NET 6 Runtime для Windows — безкоштовно завантажується з офіційного сайту Microsoft: <https://dotnet.microsoft.com/download/dotnet/6.0>;
- Microsoft Visual Studio 2022 — безкоштовна версія Community Edition доступна за посиланням: <https://visualstudio.microsoft.com/>;
- Git — система контролю версій для завантаження проєкту з репозиторію: <https://git-scm.com/>.

Для самодостатньої публікації (self-contained publish) встановлення .NET Runtime не вимагається — виконуваний файл містить усі необхідні компоненти. Розмір такого файлу складає близько 80–100 МБ.

3.2. Покрокове керівництво з інсталяції та запуску проєкту

Існує два способи запуску програми: через середовище розробки Visual Studio (для розробників) та через командний рядок .NET CLI (для кінцевих користувачів).

Спосіб 1: Запуск через Visual Studio 2022.

- 1) Завантажте та встановіть Microsoft Visual Studio 2022 Community Edition.
- 2) При встановленні оберіть робоче навантаження «.NET desktop development».
- 3) Завантажте архів проєкту DoctorHandbook.zip та розпакуйте його у зручну папку.
- 4) Відкрийте файл DoctorHandbook.csproj через меню File → Open → Project/Solution.
- 5) Переконайтеся, що у верхній панелі обрано конфігурацію «Debug» та платформу «Any CPU».
- 6) Натисніть клавішу F5 або кнопку «Start» (зелена стрілка) для запуску програми.
- 7) Visual Studio скомпілює проєкт та відкриє головне вікно «Довідник лікаря».

Спосіб 2: Запуск через .NET CLI (командний рядок).

- 1) Переконайтеся, що встановлено .NET 6 SDK (команда: dotnet --version повинна вивести 6.x.x).
- 2) Розпакуйте архів DoctorHandbook.zip.
- 3) Відкрийте командний рядок (cmd) або Windows PowerShell.
- 4) Перейдіть до папки проєкту командою: cd C:\шлях\до\DoctorHandbook
- 5) Виконайте команду запуску: dotnet run
- 6) Дочекайтеся завершення компіляції (перший запуск може тривати 10–30 секунд).
- 7) Відкриється головне вікно програми.

Команди .NET CLI:

```
# Перевірка встановленої версії .NET
dotnet --version

# Перехід до папки проекту
cd C:\Projects\DoctorHandbook

# Запуск програми
dotnet run

# Збірка виконуваного файлу
dotnet build -c Release

# Публікація самодостатнього виконуваного файлу
dotnet publish -c Release -r win-x64 --self-contained true

# Запуск зібраного файлу
.\bin\Release\net6.0-windows\win-x64\DoctorHandbook.exe
```

Після першого запуску програма автоматично завантажить тестові дані (функція SeedSampleData): три хвороби (Грип, Ангіна, Остеохондроз) та вісім медикаментів (Аспірин, Парацетамол, Ібупрофен, Амоксицилін та ін.). Ці дані зберігаються у файлах diseases.json, medicines.json та prescriptions.json у папці запуску програми.

3.3. Рекомендації з деінсталяції та резервного копіювання локальних даних

Оскільки програма «Довідник лікаря» не використовує реєстр Windows і не встановлює системних компонентів, її видалення є простим.

Деінсталяція програми:

- 1) Закрийте програму, якщо вона запущена (дані зберезуться автоматично).
- 2) Перейдіть до папки з програмою у Провіднику Windows.
- 3) Видаліть усю папку DoctorHandbook (або відповідну папку з виконуваним файлом).
- 4) Жодних додаткових дій (очищення реєстру тощо) не потрібно.

Резервне копіювання даних:

Усі дані програми зберігаються у трьох JSON-файлах у папці запуску. Для резервного копіювання достатньо скопіювати ці файли у надійне місце:

- diseases.json — база хвороб із симптомами та рекомендованими ліками;
- medicines.json — реєстр медикаментів із залишками;
- prescriptions.json — архів виписаних рецептів.

Відновлення даних: для відновлення даних скопіюйте резервні JSON-файли у папку із запущеним виконуваним файлом програми. При наступному запуску програма автоматично завантажить ці дані.

Перенесення на інший комп'ютер: для перенесення програми на інший комп'ютер достатньо скопіювати всю папку проєкту (включно з JSON-файлами даних) та встановити .NET 6 Runtime на цільовому комп'ютері. Якщо використовується самодостатня публікація (--self-contained true), встановлення .NET Runtime не потрібне.

Таблиця 3.2 — Опис JSON-файлів даних

Файл	Тип вмісту	Структура
diseases.json	Array<Disease>	[{"Id":1, "Name":"...", "Symptoms":"...", "Procedures":"...", "RecommendedMedicines":[...]}]
medicines.json	Array<Medicine>	[{"Id":1, "Name":"...", "Quantity":50, "Alternatives":"..."}]
prescriptions.json	Array<Prescription>	[{"Id":1, "PatientName":"...", "DoctorName":"...", "Date":"...", "Items":[...]}]

ВИСНОВКИ

У ході виконання курсової роботи розроблено програмний продукт «Довідник лікаря» — настільний застосунок для автоматизації щоденної клінічної роботи лікаря. Програму реалізовано мовою програмування C# з використанням технології Windows Forms на платформі .NET 6.

У першому розділі проведено ґрунтовний аналіз предметної галузі. Визначено п'ять ключових бізнес-процесів: ведення бази хвороб, облік медикаментів, формування рецептів, пошук замінників та ведення архіву. Для кожного бізнес-процесу описано проблему без автоматизації та запропоноване програмне рішення. Розроблено 14 сценаріїв використання, детально специфіковано два ключових: «Формування рецепта після огляду пацієнта» та «Коригування запасів медикаменту». Розроблено ескізи інтерфейсу всіх чотирьох вкладок програми.

У другому розділі виконано повне об'єктно-орієнтоване проектування системи. Визначено чотири пакети проєкту (Models, Data, Forms, Program) та 13 файлів з їх призначенням. Спроєктовано та описано п'ять класів-сутностей (Disease, Medicine, DiseaseMedicine, Prescription, PrescriptionItem) та клас DataStore з патерном Singleton. Для кожного класу побудовано UML-діаграму з переліком атрибутів та методів. Розроблено зведену UML-діаграму класів, що наочно демонструє зв'язки між компонентами системи. Описано алгоритм підбору замінників ліків при дефіциті з аналізом його алгоритмічної складності $O(k \cdot n \cdot a)$.

У третьому розділі підготовлено повну інструкцію з розгортання та експлуатації програми. Визначено мінімальні та рекомендовані системні вимоги. Описано два способи запуску: через Visual Studio 2022 та через .NET CLI. Наведено команди для збірки та публікації самодостатнього виконуваного файлу. Описано процедуру деінсталяції, резервного копіювання та перенесення даних на інший комп'ютер.

В результаті виконання роботи досягнуто всіх поставлених завдань:

- реалізовано базу хвороб із можливістю зберігати назву, симптоми, процедури та перелік рекомендованих ліків із кількістю;
- реалізовано базу медикаментів із управлінням залишками, пошуком за взаємозамінністю та кольоровим відображенням дефіциту;
- реалізовано функцію виписування рецептів із автоматичною перевіркою наявності ліків та їх списанням;
- реалізовано серіалізацію та десеріалізацію даних у форматі JSON з автоматичним завантаженням при запуску та збереженням при закритті;
- виконано повну валідацію введених даних у всіх формах із інформативними повідомленнями;

– дотримано принципи ООП: інкапсуляція полів моделей, єдина відповідальність класів, використання інтерфейсу IRefreshable, патерн Singleton для DataStore.

Програма повністю відповідає поставленим вимогам: реалізовано всі визначені функції, забезпечено стійкість до некоректних дій користувача (валідація форм, підтвердження видалення), збережено цілісність даних (підтвердження при відсутніх ліках). Розроблений продукт може слугувати основою для подальшого розвитку у напрямку хмарного зберігання даних, багатокористувацького доступу та розширеної аналітики медичних даних.

ПЕРЕЛІК ДЖЕРЕЛ ПОСИЛАННЯ

1. Бондарев В.М. Конспект лекцій з дисципліни «Об'єктно-орієнтоване програмування». ХНУРЕ, 2024. URL: <http://tss.co.ua:7074/> (дата звернення: 24.02.2025).
2. Common C# code conventions. Microsoft Documentation, 2024. URL: <https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/coding-conventions> (дата звернення: 24.02.2025).
3. ДСТУ 3008:2015 Інформація та документація. Звіти у сфері науки і техніки. Структура та правила оформлювання. К.: ДП «УкрНДНЦ», 2016. 26 с. URL: https://software.nure.ua/wp-content/uploads/2021/02/DSTU_3008-2015.pdf (дата звернення: 24.02.2025).
4. ДСТУ 8302:2015 Інформація та документація. Бібліографічне посилання. Загальні положення та правила складання. К.: ДП «УкрНДНЦ», 2016. 16 с. URL: <https://software.nure.ua/wp-content/uploads/2021/02/dstu-8302-2015.pdf> (дата звернення: 24.02.2025).
5. Gamma E., Helm R., Johnson R., Vlissides J. Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994. 395 p.
6. Windows Forms documentation. Microsoft Learn, 2024. URL: <https://learn.microsoft.com/en-us/dotnet/desktop/winforms/> (дата звернення: 24.02.2025).

ДОДАТОК А

Вихідний код програми (C#)

A.1 Клас Disease (Models/Disease.cs)

```
using System;
using System.Collections.Generic;

namespace DoctorHandbook.Models
{
    /// <summary>
    /// Описує медичне захворювання з переліком симптомів,
    /// процедур та рекомендованих медикаментів.
    /// </summary>
    [Serializable]
    public class Disease
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public string Symptoms { get; set; }
        public string Procedures { get; set; }
        public List<DiseaseMedicine> RecommendedMedicines { get; set; }

        public Disease()
        {
            RecommendedMedicines = new List<DiseaseMedicine>();
        }

        public Disease(int id, string name, string symptoms, string
procedures)
        {
            Id = id; Name = name;
            Symptoms = symptoms; Procedures = procedures;
            RecommendedMedicines = new List<DiseaseMedicine>();
        }

        public override string ToString() => Name;
    }
}
```

A.2 Клас Medicine (Models/Medicine.cs)

```
namespace DoctorHandbook.Models
{
    /// <summary>
    /// Описує медикамент на складі з кількістю та
```

```

/// переліком взаємозамінних препаратів.
/// </summary>
[Serializable]
public class Medicine
{
    public int Id { get; set; }
    public string Name { get; set; }
    public int Quantity { get; set; }
    public string Alternatives { get; set; }

    public Medicine() { }

    public Medicine(int id, string name, int qty, string alt)
    {
        Id = id; Name = name; Quantity = qty; Alternatives = alt;
    }

    public override string ToString()
        => $"{Name} (в наявності: {Quantity})";
}
}

```

A.3 Клас DataStore (Data/DataStore.cs) — ключові методи

```

using System.Text.Json;
using DoctorHandbook.Models;

namespace DoctorHandbook.Data
{
    /// <summary>
    /// Singleton-сховище даних. Забезпечує CRUD-операції
    /// та серіалізацію/десеріалізацію у JSON.
    /// </summary>
    public class DataStore
    {
        private static DataStore _instance;
        public static DataStore Instance
            => _instance ?? (_instance = new DataStore());

        private DataStore()
        {
            Diseases      = new List<Disease>();
            Medicines      = new List<Medicine>();
            Prescriptions = new List<Prescription>();
        }

        public List<Disease>      Diseases      { get; private set; }
        public List<Medicine>     Medicines     { get; private set; }
        public List<Prescription> Prescriptions { get; private set; }
    }
}

```



```

// --- Medicines ---
public bool AdjustMedicineStock(int id, int qty)
{
    var med = Medicines.Find(m => m.Id == id);
    if (med == null || med.Quantity < qty) return false;
    med.Quantity -= qty;
    return true;
}

// --- Serialization ---
public void SaveAll()
{
    var opt = new JsonSerializerOptions { WriteIndented = true
};
    File.WriteAllText("diseases.json",
        JsonSerializer.Serialize(Diseases, opt));
    File.WriteAllText("medicines.json",
        JsonSerializer.Serialize(Medicines, opt));
    File.WriteAllText("prescriptions.json",
        JsonSerializer.Serialize(Prescriptions, opt));
}
}

```

ДОДАТОК Б

Графічні матеріали та розширені блок-схеми алгоритмів

Б.1 Блок-схема алгоритму виписування рецепта

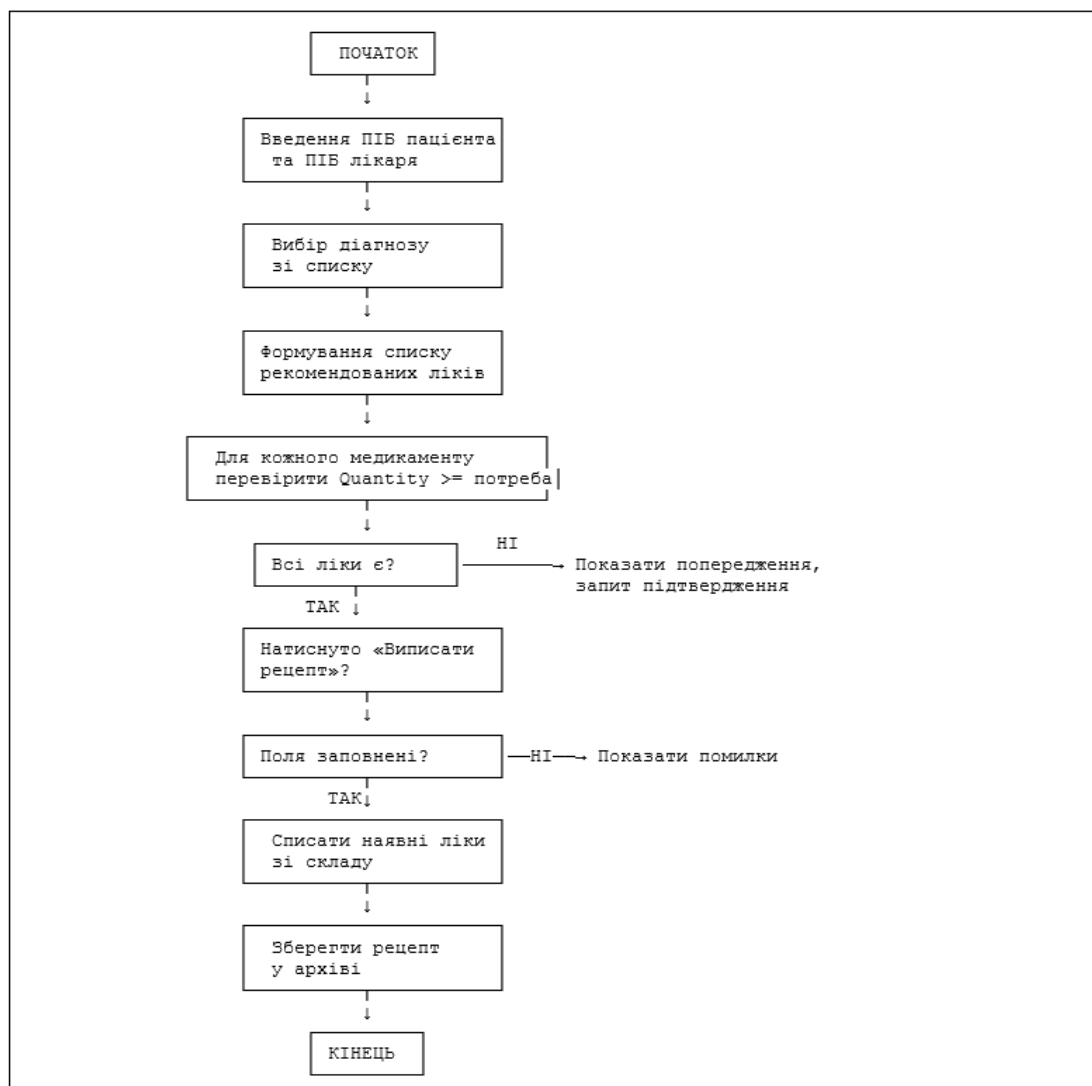


Рисунок Б.1 — Блок-схема алгоритму виписування рецепта

Б.2 Блок-схема алгоритму збереження та завантаження даних

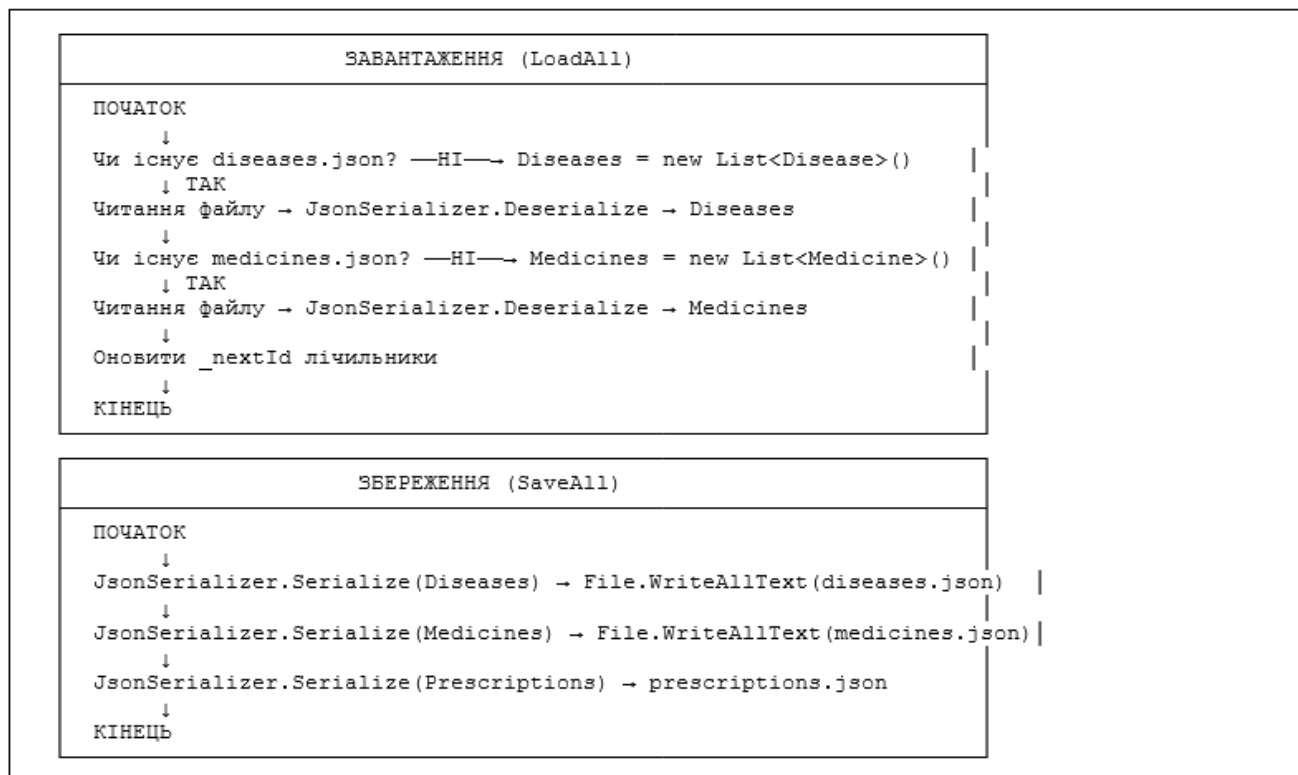


Рисунок Б.2 — Блок-схема алгоритму збереження та завантаження даних

Посилання на Github: <https://github.com/mariaryzhova-wq/oopro.doctor>